

Розділ 1. Аналітичний огляд

Виконали: Студенти групи ІО-41 Вовк Андрій та Троценко Максим

1.1 Опис предметної області

Обрана тема належить до предметної області аналізу файлових систем і керування дисковим простором. У процесі роботи з персональним комп'ютером або сервером на накопичувачах поступово накопичується велика кількість файлів різних типів: документи, зображення, відео, архіви, програмний код, тимчасові файли та системні дані. Через це користувачу часто складно швидко визначити, які саме директорії або файли займають найбільше місця, які дані можна видалити, а які варто залишити.

Програмні засоби аналізу файлової системи призначені для сканування диска або окремої директорії, збору інформації про файли та папки, визначення їх розміру, побудови ієрархічної структури каталогів і візуалізації використання дискового простору. Такі системи дозволяють користувачу швидше знаходити великі файли, аналізувати розподіл даних за типами, сортувати результати сканування та виконувати базові дії з файлами.

Для системи File System Analyzer основними об'єктами аналізу є файли, директорії та символічні посилання. Для кожного об'єкта доцільно зберігати шлях, назву, розширення, тип і розмір. Додатково для директорій необхідно обчислювати сумарний розмір усіх вкладених файлів і папок, що дозволяє оцінити частку кожного піддерева у загальному обсязі просканованих даних.

1.2 Аналіз існуючих програмних рішень

На сьогодні існує багато програмних засобів для аналізу використання дискового простору. Вони відрізняються типом інтерфейсу, швидкістю роботи, підтримкою фільтрації, можливостями візуалізації та доступністю на різних операційних системах.

Одним із поширених рішень є WinDirStat. Це графічна програма для Windows, яка сканує файлову систему, показує дерево директорій і візуалізує зайнятий простір за допомогою спеціальної карти файлів. Її перевагою є наочність, проте програма орієнтована переважно на Windows і не є зручною для роботи в терміналі.

TreeSize Free також використовується для аналізу зайнятого місця на диску. Програма має зручний графічний інтерфейс, дозволяє переглядати розміри директорій і сортувати їх за обсягом. Водночас безкоштовна версія має обмежену функціональність, а сама програма більше орієнтована на користувачів Windows.

У Linux-подібних системах часто використовується утиліта `ncdu`. Вона працює в терміналі, має зручний текстовий інтерфейс і дозволяє швидко переглядати розміри директорій. Її недоліком є обмежені можливості аналітики за типами файлів і відсутність розширеної візуальної статистики.

Ще одним аналогом є Baobab, або Disk Usage Analyzer, який використовується в середовищі GNOME. Програма має графічний інтерфейс і дозволяє візуально аналізувати використання диска. Однак вона залежить від графічного середовища і не підходить для сценаріїв, де потрібна легка термінальна програма.

Також існує стандартна консольна утиліта `du`, яка дозволяє отримувати інформацію про розмір файлів і директорій. Вона є простою та доступною майже в усіх Unix-подібних

системах, але не має інтерактивного інтерфейсу, дерева навігації, фільтрації, прогресу сканування та статистики за категоріями файлів.

1.3 Порівняння аналогів

Програмне рішення	Тип інтерфейсу	Платформа	Дерево каталогів	Фільтрація	Статистика
WinDirStat	GUI	Windows	Так	Обмежена	Візуальна карта файлів
TreeSize Free	GUI	Windows	Так	Обмежена	Базова статистика
ncdu	TUI	Linux, macOS	Так	Базова	Обмежена
Baobab	GUI	Linux	Так	Обмежена	Графічна візуалізація
du	CLI	Unix-подібні ОС	Ні	Через параметри команди	Немає

З порівняння видно, що існуючі рішення здебільшого спеціалізуються або на графічному аналізі дискового простору, або на простому консольному підрахунку розмірів. Не всі аналоги поєднують інтерактивний термінальний інтерфейс, зручну навігацію деревом директорій, фільтрацію за розширенням і розміром, статистику за типами файлів та безпечну обробку помилок доступу.

1.4 Недоліки існуючих рішень

Основними недоліками розглянутих аналогів є:

- залежність частини програм від конкретної операційної системи;
- орієнтація багатьох рішень на графічний інтерфейс, що не завжди зручно для швидкої роботи в терміналі;
- відсутність або обмеженість фільтрації за типом, розширенням чи мінімальним розміром файлів;
- недостатня деталізація статистики за категоріями файлів;
- відсутність прогресу сканування та поточної інформації про кількість знайдених файлів у простих консольних утилітах;
- обмежені можливості інтерактивної роботи з результатами сканування;
- недостатньо гнучка обробка помилок доступу у випадку закритих або системних директорій.

Зазначені недоліки обґрунтовують необхідність розробки власної системи, яка буде поєднувати переваги консольних і графічних засобів аналізу файлової системи.

1.5 Обґрунтування необхідності розробки власної системи

Розробка власної системи File System Analyzer є доцільною, оскільки вона дозволяє реалізувати набір функцій, орієнтований на конкретні потреби користувача. Майбутня система повинна виконувати рекурсивне сканування директорій, збирати метадані про файли та папки, обчислювати розмір піддерев, сортувати та фільтрувати результати, а також відображати інформацію у зручному термінальному інтерфейсі.

Важливою перевагою власного рішення є можливість реалізації безпечної обробки помилок доступу. Якщо під час сканування програма не має прав на читання певної директорії, вона не повинна завершувати роботу аварійно, а має пропустити недоступний об'єкт і продовжити аналіз інших даних.

Крім того, система може містити модуль статистики, який групує файли за категоріями, наприклад відео, аудіо, документи, архіви або програмний код. Це дозволить користувачу не лише бачити загальний розмір директорій, а й розуміти структуру зайнятого простору.

Таким чином, власна система File System Analyzer має поєднати швидке сканування файлової системи, інтерактивну навігацію, сортування, фільтрацію, статистичний аналіз і базові дії з файлами. Це робить її актуальною та практично корисною для аналізу використання дискового простору.

Розділ 2. Проектування системи

2.1 Загальний підхід до проектування

Під час проектування системи File System Analyzer було обрано модульний підхід. Такий підхід є доцільним, оскільки функціональність системи логічно поділяється на окремі частини: сканування файлової системи, збір метаданих, обчислення розмірів директорій, фільтрацію, сортування, формування статистики, взаємодію з користувачем і збереження результатів у базі даних.

Система проектується як термінальний застосунок з інтерактивним TUI-інтерфейсом. Користувач запускає програму, обирає директорію для аналізу, після чого система виконує рекурсивне сканування файлової структури, збирає інформацію про знайдені файли та папки, обчислює сумарні розміри директорій і відображає результат у вигляді дерева. Додатково користувач може застосовувати сортування, фільтри, переглядати статистику та виконувати базові дії з файлами.

Основною мовою реалізації обрано Rust, оскільки вона забезпечує високу швидкість, безпечну роботу з пам'яттю та зручні засоби для розробки консольних застосунків. Для збереження результатів сканування доцільно використовувати SQLite, оскільки ця СУБД не потребує окремого серверного процесу, є простою у використанні та достатньою для локального застосунку.

2.2 Архітектура системи

Архітектура системи складається з трьох основних рівнів:

- рівень представлення, який відповідає за TUI-інтерфейс і взаємодію з користувачем;
- рівень бізнес-логіки, який реалізує сканування, обробку метаданих, сортування, фільтрацію та статистику;
- рівень збереження даних, який відповідає за роботу з базою даних і збереження результатів сканування.

Загальну структуру системи наведено у вигляді логічної схеми.

Рівень представлення TUI-інтерфейс Навігація Відображення дерева	Рівень бізнес-логіки Сканування Метадані Сортування Фільтрація Статистика	Рівень даних SQLite Історія сканувань Файли та папки Статистика
---	--	---

Table 1: Логічна архітектура системи File System Analyzer

Такий поділ дозволяє зменшити залежність між частинами програми. Наприклад, зміна способу збереження даних не повинна впливати на логіку сканування, а зміна інтерфейсу не повинна вимагати переробки структури бази даних.

2.3 Основні функціональні модулі

До складу системи входять такі функціональні модулі:

- **Модуль TUI-інтерфейсу** відповідає за відображення дерева директорій, прогресу сканування, повідомлень про помилки, панелі статистики та обробку команд користувача.
- **Модуль сканування файлової системи** виконує рекурсивний обхід директорій, визначає типи об'єктів файлової системи та передає знайдені дані на подальшу обробку.
- **Модуль збору метаданих** отримує шлях, назву, розширення, розмір і тип кожного знайденого об'єкта.
- **Модуль обчислення розмірів** виконує підрахунок сумарного розміру директорій на основі розмірів вкладених файлів і підкаталогів.
- **Модуль сортування** забезпечує впорядкування результатів за назвою, розміром або типом.
- **Модуль фільтрації** дозволяє відображати лише ті файли та директорії, які відповідають заданим умовам, наприклад певному розширенню або мінімальному розміру.
- **Модуль статистики** групує файли за категоріями та обчислює відсотковий розподіл зайнятого простору.
- **Модуль отримання системної інформації** визначає загальний, вільний і використаний обсяг диска для порівняння з результатами сканування.
- **Модуль роботи з базою даних** зберігає інформацію про сканування, знайдені файли, помилки доступу та сформовану статистику.
- **Модуль інтерактивних дій** відповідає за видалення файлів або папок з підтвердженням, відкриття об'єкта у файловому менеджері та скасування процесу сканування.

2.4 UML-діаграма прецедентів

Основним актором системи є користувач. Він взаємодіє з програмою через термінальний інтерфейс, запускає сканування, переглядає результати, застосовує фільтри та виконує інтерактивні дії.

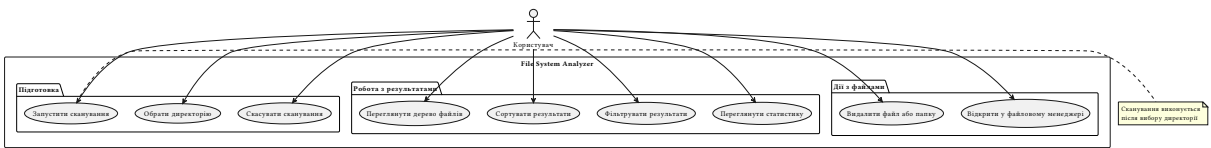


Figure 1: UML-діаграма прецедентів системи File System Analyzer

Діаграма прецедентів показує основні можливості системи. Спочатку користувач обирає директорию, після чого запускає сканування. Після завершення сканування користувач отримує доступ до дерева файлів, статистики, сортування, фільтрації та додаткових дій.

2.5 UML-діаграма класів

Оскільки система реалізується мовою Rust, замість класів у програмному коді використовуватимуться структури, переліки та модулі. Проте для проєктування їх можна представити у вигляді UML-діаграми класів.

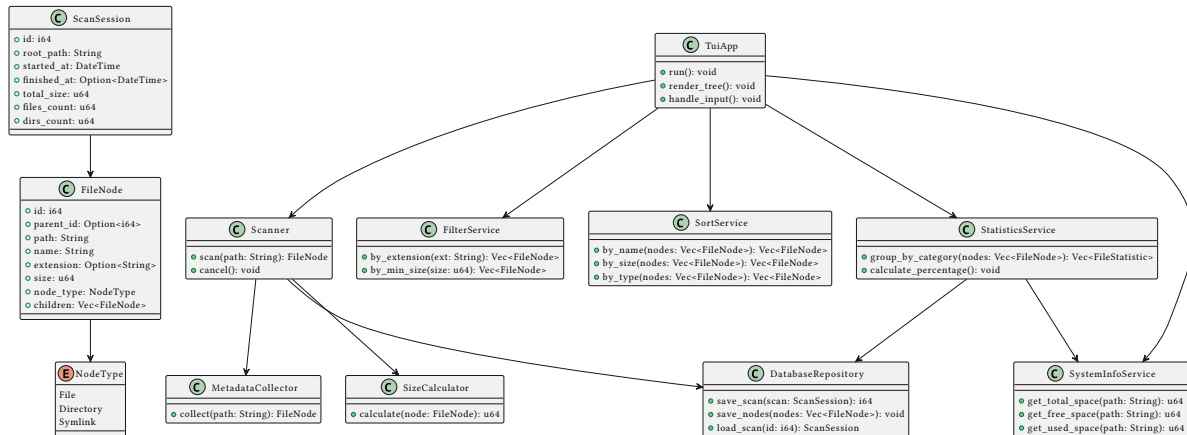


Figure 2: UML-діаграма класів системи File System Analyzer

Ключовою структурою є `FileNode`, яка описує елемент файлової системи. Для файлу вона містить фактичний розмір, а для директорії - сумарний розмір усіх вкладених елементів. Структура `ScanSession` описує окремий запуск сканування та використовується для збереження історії роботи програми. Окремий сервіс `SystemInfoService` відповідає за отримання інформації про загальний, вільний і використаний обсяг диска.

2.6 UML-діаграма послідовності

Нижче наведено діаграму послідовності для основного сценарію роботи системи: запуск сканування директорії та відображення результатів користувачу.

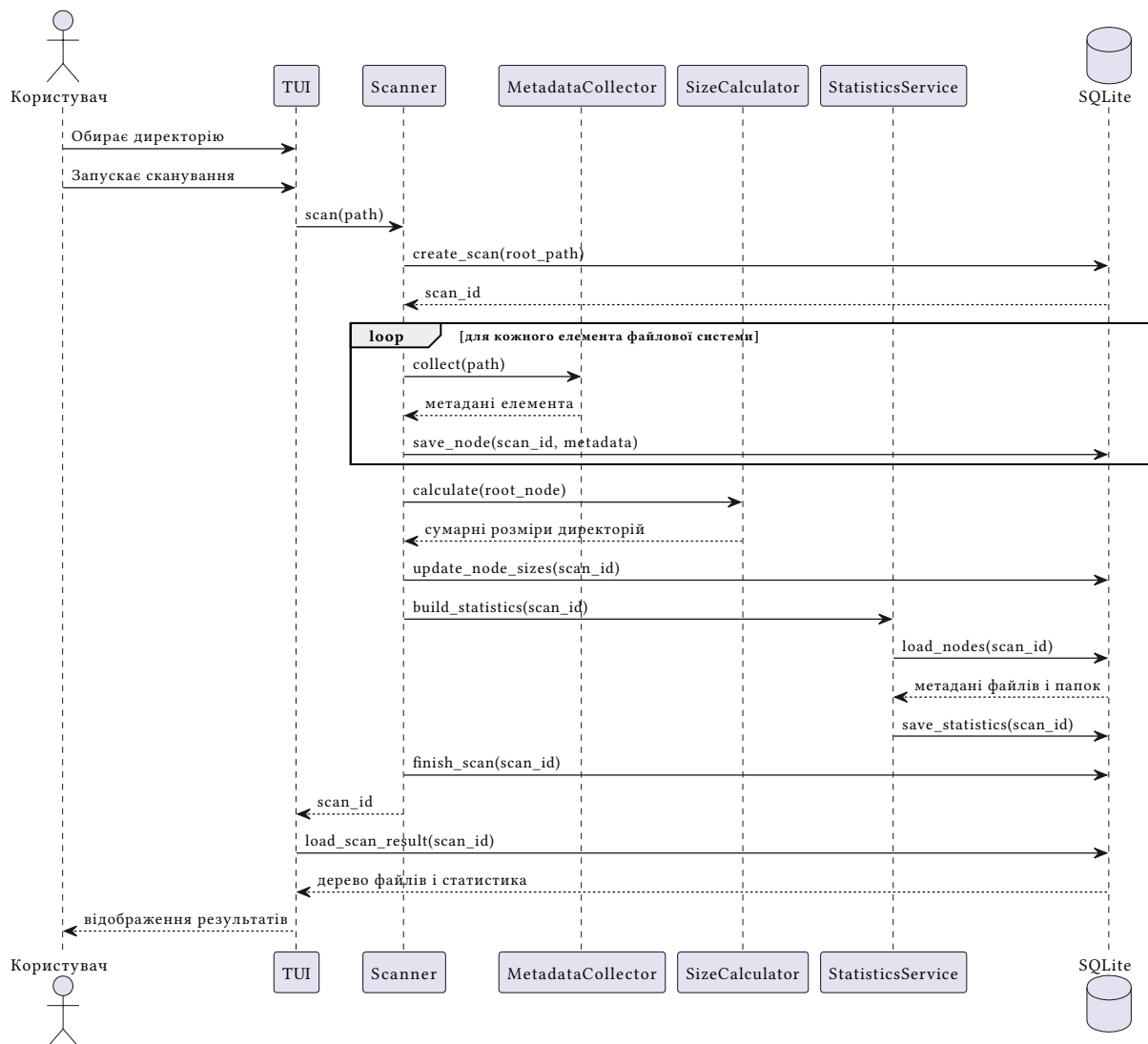


Figure 3: UML-діаграма послідовності процесу сканування директорії

Цей сценарій показує, що після вибору директорії користувач окремо запускає сканування. Модуль сканування збирає метадані про файли та папки, після чого зберігає їх у базі даних SQLite. Після обчислення розмірів і формування статистики TUI-модуль отримує підготовлені результати з бази даних і відображає їх користувачу.

2.7 Структура бази даних

Для збереження результатів роботи системи пропонується використовувати реляційну базу даних SQLite. База даних потрібна для збереження історії сканувань, структури знайдених файлів і папок, статистики за типами файлів та інформації про помилки доступу. Це дозволяє переглядати попередні результати без повторного сканування файлової системи.

Основними таблицями бази даних є scans, file_nodes, file_statistics і scan_errors.

2.7.1 Таблиця scans

Таблиця scans зберігає інформацію про кожен запуск сканування.

Поле	Тип	Опис
id	INTEGER PRIMARY KEY	Унікальний ідентифікатор сканування

Поле	Тип	Опис
root_path	TEXT NOT NULL	Початкова директорія, обрана користувачем
started_at	DATETIME NOT NULL	Дата та час початку сканування
finished_at	DATETIME	Дата та час завершення сканування
total_size	INTEGER	Загальний розмір просканованих даних у байтах
files_count	INTEGER	Кількість знайдених файлів
dirs_count	INTEGER	Кількість знайдених директорій
status	TEXT	Стан сканування: completed, cancelled або failed

2.7.2 Таблиця file_nodes

Таблиця file_nodes зберігає інформацію про знайдені файли, директорії та символічні посилання.

Поле	Тип	Опис
id	INTEGER PRIMARY KEY	Унікальний ідентифікатор елемента
scan_id	INTEGER NOT NULL	Зовнішній ключ на scans.id
parent_id	INTEGER	Зовнішній ключ на батьківський запис у file_nodes.id
path	TEXT NOT NULL	Повний шлях до елемента файлової системи
name	TEXT NOT NULL	Назва файлу або директорії
extension	TEXT	Розширення файлу
size	INTEGER NOT NULL	Розмір у байтах
node_type	TEXT NOT NULL	Тип елемента: file, directory або symlink
percentage	REAL	Частка від розміру батьківської директорії

Зв'язок parent_id дозволяє зберігати ієрархічну структуру файлової системи у реляційній базі даних. Для кореневої директорії значення parent_id дорівнює NULL.

2.7.3 Таблиця file_statistics

Таблиця file_statistics використовується для збереження агрегованої статистики за категоріями файлів.

Поле	Тип	Опис
id	INTEGER PRIMARY KEY	Унікальний ідентифікатор запису
scan_id	INTEGER NOT NULL	Зовнішній ключ на scans.id
category	TEXT NOT NULL	Категорія файлів: video, audio, documents, code, archives тощо

Поле	Тип	Опис
files_count	INTEGER	Кількість файлів у категорії
total_size	INTEGER	Сумарний розмір файлів категорії у байтах
percentage	REAL	Відсоток від загального обсягу просканиваних даних

2.7.4 Таблиця scan_errors

Таблиця scan_errors зберігає інформацію про помилки, які виникли під час сканування.

Поле	Тип	Опис
id	INTEGER PRIMARY KEY	Унікальний ідентифікатор помилки
scan_id	INTEGER NOT NULL	Зовнішній ключ на scans.id
path	TEXT NOT NULL	Шлях до об'єкта, під час обробки якого виникла помилка
error_kind	TEXT NOT NULL	Тип помилки, наприклад permission_denied
message	TEXT	Текстове повідомлення про помилку

2.8 Зв'язки між таблицями

Між таблицями бази даних передбачено такі зв'язки:

- один запис у таблиці scans може мати багато записів у таблиці file_nodes;
- один запис у таблиці scans може мати багато записів у таблиці file_statistics;
- один запис у таблиці scans може мати багато записів у таблиці scan_errors;
- таблиця file_nodes має самозв'язок через поле parent_id, що дозволяє описати дерево директорій.

Поле id у таблиці scans є єдиним ідентифікатором сканування. Поле scan_id у таблицях file_nodes, file_statistics і scan_errors не створює нове сканування, а лише посилається на відповідний запис у таблиці scans як зовнішній ключ.

Умовну ER-структуру бази даних можна подати так:

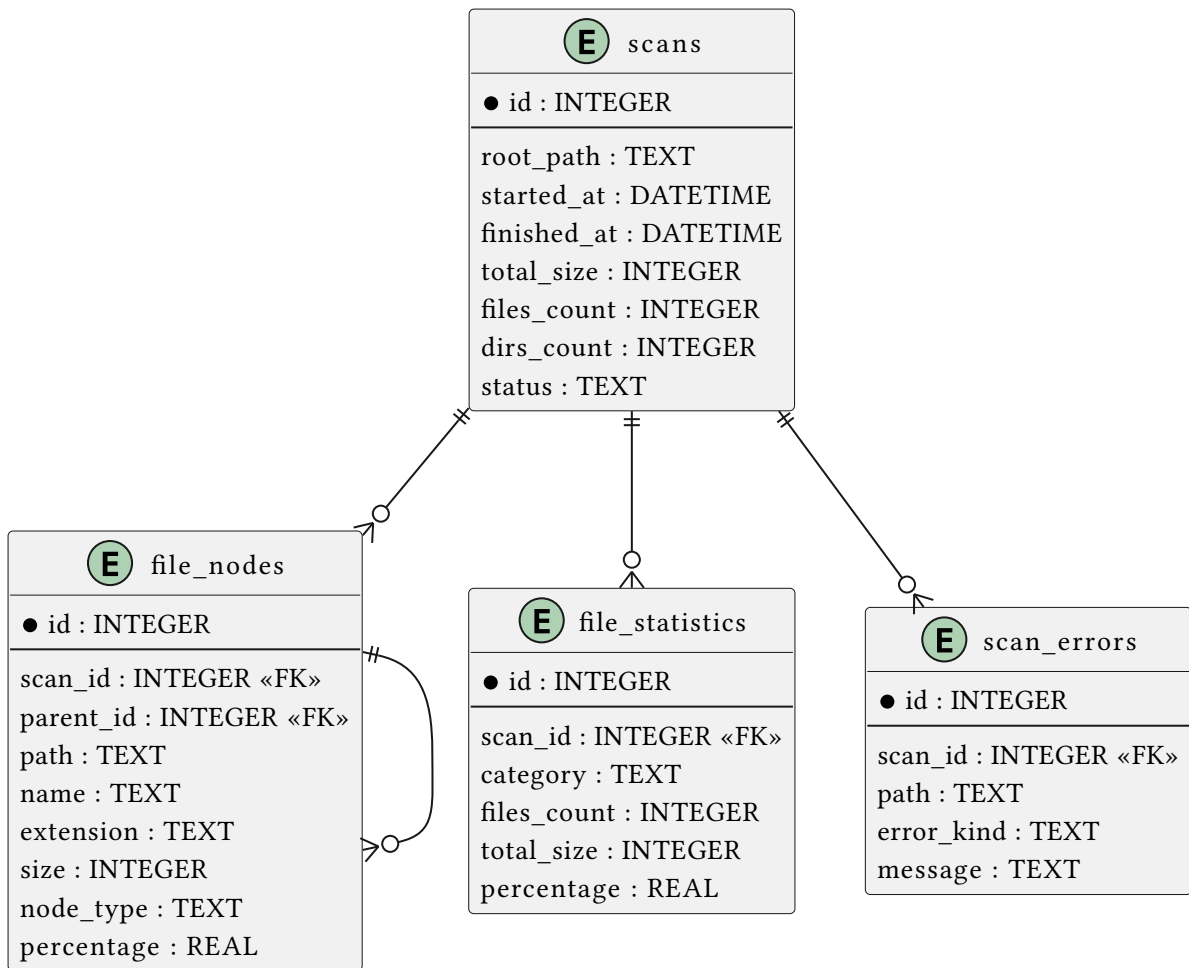


Figure 4: ER-структура бази даних системи File System Analyzer

2.9 Висновки до розділу

У другому розділі було визначено підхід до проектування системи File System Analyzer. Для майбутньої системи обрано модульну архітектуру з поділом на рівень представлення, рівень бізнес-логіки та рівень збереження даних. Було визначено основні функціональні модулі, підготовлено UML-діаграми прецедентів, класів і послідовності, а також описано структуру бази даних SQLite.

Запропонована структура системи забезпечує можливість рекурсивного сканування файлової системи, збереження результатів аналізу, побудови дерева директорій, виконання сортування та фільтрації, формування статистики й обробки помилок доступу без аварійного завершення програми.